

1 SVD: Scratch Code and TensorFlow Implementation

Problem Statement

Natural images tend to concentrate most of their information in a small number of dominant singular values, keeping only the top- k components gives a low-rank approximation A_k that reconstructs the image with little visible loss while discarding the rest. This makes SVD well suited to image compression: this section builds the decomposition from scratch on a grayscale image using power iteration with deflation (to recover eigenpairs of $A^T A$ one at a time without needing a full eigendecomposition library), then verifies the result against TensorFlow's built-in `tf.linalg.svd`, comparing reconstruction error (MSE) and % variance retained between the two. The full scratch implementation is available at `svd.ipynb`.

Scratch Code

Loading and Preprocessing the Image

```
img_gray = img.convert('L')
img_resized = img_gray.resize(RESIZE_DIM)
image_matrix = np.array(img_resized, dtype=np.float64) / 255.0
```

`convert('L')` collapses a color image down to a single grayscale channel, computed as a weighted sum of the red, green, and blue channels, $L = 0.299R + 0.587G + 0.114B$, the standard luminance weighting. This is necessary so the image can be treated as one 2D matrix; a color image would otherwise be three separate $m \times n$ channels, each needing its own decomposition. `resize(RESIZE_DIM)` fixes the matrix to a known, controlled size, here 128×128 , which keeps the power iteration computations below manageable in cost. Dividing by 255 rescales pixel intensities, originally 8-bit integers in $[0, 255]$, down to $[0, 1]$, normalization for standardizing inputs: it keeps the numbers in a well-behaved range for the linear algebra that follows and makes later energy/variance ratios cleanly interpretable. The result is

$$A \in \mathbb{R}^{128 \times 128}, \quad A_{pq} = \frac{\text{pixel}_{pq}}{255} \in [0, 1].$$

Forming $A^T A$ and AA^T

```
ATA = np.dot(A.T, A)
AAT = np.dot(A, A.T)
```

Starting from the SVD definition $A = U\Sigma V^T$, with U, V orthonormal ($U^T U = I$, $V^T V = I$):

$$A^T A = V \Sigma^T U^T U \Sigma V^T = V \Sigma^2 V^T, \quad AA^T = U \Sigma V^T V \Sigma^T U^T = U \Sigma^2 U^T.$$

Both are eigendecomposition equations: the eigenvectors of $A^T A$ are exactly the right singular vectors v_i , with eigenvalues σ_i^2 , while the eigenvectors of AA^T are the left singular vectors u_i , also with eigenvalues σ_i^2 . So in principle either matrix could be used to recover one half of the SVD. `AAT` is computed here but never actually used in the scratch pipeline, because U ends up being recovered directly from A and V instead (see below), which avoids running a second, independent power-iteration-based eigendecomposition, and avoids any risk of the two separately computed eigenbases being inconsistent with each other.

Power Iteration

```
b_k = np.random.rand(n)
b_k = b_k / np.linalg.norm(b_k)
for i in range(num_iterations):
    b_k1 = np.dot(matrix, b_k)
    b_k1_norm = np.linalg.norm(b_k1)
    b_k_new = b_k1 / b_k1_norm
    if np.linalg.norm(b_k_new - b_k) < tol:
        b_k = b_k_new
        break
    b_k = b_k_new
eigenvalue = np.dot(b_k.T, np.dot(matrix, b_k)) / np.dot(b_k.T, b_k)
```

The update rule repeatedly applies the matrix and renormalizes:

$$b_{t+1} = \frac{Mb_t}{\|Mb_t\|}.$$

To see why this converges to the dominant eigenvector, write the random starting vector in the eigenbasis of M (symmetric here, so an orthonormal eigenbasis exists), $b_0 = \sum_i c_i v_i$. Then $M^t b_0 = \sum_i c_i \lambda_i^t v_i$. Factoring out λ_1^t , the largest eigenvalue, leaves $c_1 v_1 + \sum_{i>1} c_i (\lambda_i/\lambda_1)^t v_i$, and since $|\lambda_i/\lambda_1| < 1$ for every $i > 1$, those terms shrink to zero as t grows, leaving only the direction of v_1 . So normalizing at every step keeps the vector bounded while its direction converges toward the dominant eigenvector:

$$b_t \longrightarrow v_1 \quad (\text{dominant eigenvector of } M).$$

The convergence check, $\|b_k^{\text{new}} - b_k\| < \text{tol}$, measures how much the direction is still shifting between consecutive iterations; once it stabilizes below the tolerance, iteration stops early rather than running the full `num_iterations`. Once b_k has (near) converged and is unit-norm, the eigenvalue is recovered via the Rayleigh quotient

$$\lambda_1 = \frac{b_k^T M b_k}{b_k^T b_k},$$

which is exact whenever b_k is precisely an eigenvector, and a close estimate otherwise; with $b_k^T b_k \approx 1$ the denominator is essentially just a normalization safeguard.

Deflation Loop

```
for i in range(k):
    eigenvalue_v, eigenvector_v = power_iteration(current_matrix_for_v, ...)
    sigma_i = np.sqrt(abs(eigenvalue_v))
    singular_values_list.append(sigma_i)
    V_scratch[:, i] = eigenvector_v
    current_matrix_for_v -= eigenvalue_v * np.outer(eigenvector_v, eigenvector_v)
```

Power iteration on its own always converges to the same dominant eigenvector, run it twice and you get v_1 again, not v_2 . Deflation solves this: once λ_i, v_i are found, the rank-one component along v_i is subtracted out of the matrix,

$$M \leftarrow M - \lambda_i v_i v_i^T.$$

Because the eigenbasis of a symmetric matrix is orthogonal, this subtraction leaves every other eigenvector $v_j, j \neq i$, with its eigenvalue λ_j untouched, while v_i 's own eigenvalue in the deflated matrix drops to zero. So the next call to `power_iteration` on this deflated matrix now converges to what was previously the second-largest eigenpair, and the process repeats. Each eigenvalue found is converted to a singular value via

$$\sigma_i = \sqrt{\lambda_i},$$

since by definition the eigenvalues of $A^T A$ are σ_i^2 . Running this loop k times builds up $v_1, \dots, v_k$ and $\sigma_1, \dots, \sigma_k$, in approximately, but not strictly guaranteed, descending order, since power iteration is an approximate iterative method and its convergence speed can vary slightly from one deflation round to the next.

Recovering U from V

```
U_scratch[:, i] = np.dot(A, V_scratch[:, i]) / singular_values_list[i]
```

Starting from $A = U \Sigma V^T$ and multiplying both sides on the right by v_i , using $V^T v_i = e_i$ (orthonormality), gives $A v_i = \sigma_i u_i$, rearranged:

$$u_i = \frac{1}{\sigma_i} A v_i.$$

So once a right singular vector v_i and its singular value σ_i are known, the corresponding left singular vector u_i falls out of a single matrix-vector multiplication and a scalar division, no separate eigenproblem on $A A^T$ required. This is exactly why `AAT`, computed earlier, never gets used: it would have been a redundant, independent route to the same U .

Sorting

```
sorted_indices = np.argsort(singular_values_list)[::-1]
singular_values_sorted = np.array(singular_values_list)[sorted_indices]
U_scratch = U_scratch[:, sorted_indices]
V_scratch = V_scratch[:, sorted_indices]
```

`np.argsort` returns the indices that would sort the singular values in ascending order; reversing with `[::-1]` flips this to descending order, the standard SVD convention $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_k$. The same index ordering is then applied to reindex the columns of both `U_scratch` and `V_scratch`, so each triplet (σ_i, u_i, v_i) stays correctly matched, just reordered. This step is a safeguard rather than a core mathematical operation: because power iteration with finite `num_iterations` and `tol` is only approximate, and successive deflation rounds can converge at slightly different rates, the raw order in which singular values are discovered can occasionally drift out of strict descending order, this line enforces the canonical ordering regardless.

Reconstruction

```
S_reconstruct_scratch = S_scratch[:, :k]
reconstructed_matrix_scratch = U_scratch[:, :k] @ S_reconstruct_scratch @ Vt_scratch[:, :]
```

The truncated SVD reconstruction keeps only the top k singular triplets:

$$A_k = U_k \Sigma_k V_k^T = \sum_{i=1}^k \sigma_i u_i v_i^T.$$

Shape-wise, $U_k \in \mathbb{R}^{128 \times k}$, $\Sigma_k \in \mathbb{R}^{k \times k}$ diagonal, $V_k^T \in \mathbb{R}^{k \times 128}$; multiplying these three back together returns a matrix of the original shape, 128×128 , but built entirely from k rank-one outer products rather than the full set of $\min(128, 128) = 128$ available directions. This is the core mechanism behind SVD-based image compression: storing only U_k , $\sigma_1, \dots, \sigma_k$, and V_k , far fewer numbers than the full 128×128 matrix when k is small, is enough to approximately reconstruct the image.

Reconstruction Error (MSE)

```
mse_scratch = np.mean((original_flat - reconstructed_flat_scratch)**2)
```

$$\text{MSE} = \frac{1}{mn} \sum_{p,q} (A_{pq} - (A_k)_{pq})^2.$$

This is the average squared difference, pixel by pixel, between the original image and its rank- k approximation. Smaller k discards more of the singular value spectrum and generally produces higher MSE, while as k approaches the full rank of A , $A_k \rightarrow A$ and MSE approaches zero.

Variance Retained

```
total_variance_scratch = np.sum(image_matrix**2)
retained_variance_scratch = np.sum(np.array(singular_values_for_plot)**2)
variance_retained_scratch = (retained_variance_scratch / total_variance_scratch) * 100
```

The denominator here is a shortcut: the squared Frobenius norm of A equals the sum of squares of every singular value, not just the top k ,

$$\|A\|_F^2 = \sum_{p,q} A_{pq}^2 = \text{trace}(A^T A) = \sum_{\text{all } i} \sigma_i^2,$$

since trace is basis-independent and equals the sum of $A^T A$'s eigenvalues, which are exactly the σ_i^2 . So `total_variance_scratch` is computed directly from the raw pixel values, $\sum_{p,q} A_{pq}^2$, sidestepping the need to deflate the power iteration all the way down to full rank just to obtain every singular value. `retained_variance_scratch` sums the squares of only the k singular values actually computed, giving

$$\% \text{ Variance Retained} = \frac{\sum_{i=1}^k \sigma_i^2}{\|A\|_F^2} \times 100,$$

the percentage of the image's total energy captured by the rank- k approximation, higher meaning a more faithful compression at that k .

Output

```
Configuration:
Image Filename: apple.jpeg
Resize Dimension: (128, 128)
Top-K Singular Values for Reconstruction: 20
Original image format: JPEG, size: (1880, 1204), mode: L
Resizing image to 128x128 pixels.
Processed image matrix shape: (128, 128)
Min pixel value: 0.0667, Max pixel value: 0.8784
Starting SVD from scratch for 20 components...

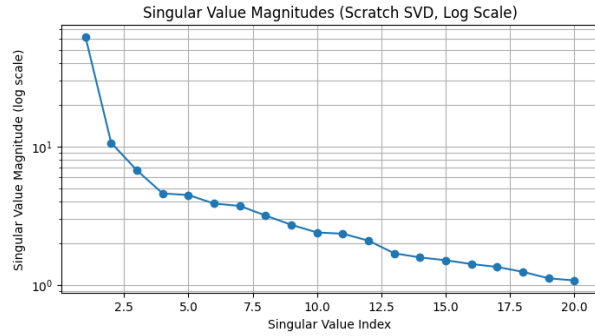
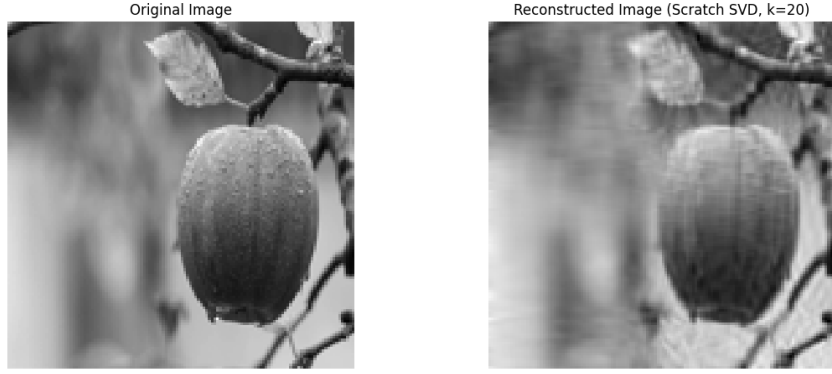
===== SCRIPT 1: SVD FROM SCRATCH =====
Step 1: Compute A.T @ A and A @ A.T
Shape of A.T @ A: (128, 128)
Shape of A @ A.T: (128, 128)
Step 2: Extracting eigenvectors/eigenvalues of A.T @ A using power iteration with deflation.
Processing for 1-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 2-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 3-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 4-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 5-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 6-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 7-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 8-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 9-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 10-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 11-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 12-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 13-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 14-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 15-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 16-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 17-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 18-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 19-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Processing for 20-th singular value/vector...
Starting power iteration for a 128x128 matrix...
Step 3: Deriving U vectors.
Top 20 Singular Values (from A.T @ A): [62.01269075 10.61735207 6.711951 4.5700015 4.44934598]...
Shape of U_scratch: (128, 20)
Shape of V_scratch: (128, 20)

Step 4: Reconstructing image using top-k singular values/vectors.
Shape of reconstructed matrix: (128, 128)

Step 5: Computing reconstruction error (MSE) and % variance retained.
Mean Squared Error (MSE) for scratch SVD reconstruction: 0.000579
% Variance Retained (scratch SVD, k=20): 99.77%

Step 6: Plotting results.
```

Variance Retained: 99.77%



Actual VS Predicted Output

TensorFlow Version

Uses the same preprocessed image matrix (`image_matrix`) and `TOP_K_SINGULAR_VALUES` configuration as the scratch version above; no data loading or preprocessing is repeated here.

`tf.linalg.svd`

```
s, U_tf, V_tf = tf.linalg.svd(image_tensor)
Vt_tf_np = V_tf_np.T
```

This computes the exact same factorization, $A = U\Sigma V^T$, but via a single optimized library call rather than the scratch implementation's iterative, one-eigenpair-at-a-time power iteration plus deflation. Internally this relies on established numerical linear algebra routines (typically bidiagonalization followed by an iterative diagonalization step), and it returns the *entire* singular value spectrum at once, $s = (\sigma_1, \dots, \sigma_{\min(m,n)})$, along with full U and V matrices, rather than only the top k that the scratch loop deliberately stops at. One bookkeeping difference: `tf.linalg.svd` returns V itself, not V^T , so an explicit transpose is needed to match the $U\Sigma V^T$ convention used in reconstruction.

Reconstruction and Variance Retained

```
reconstructed_matrix_tf = U_k_tf @ S_k_tf @ Vt_k_tf
total_variance_tf = np.sum(s_np**2)
variance_retained_tf = np.sum(s_np[:k]**2) / total_variance_tf
```

The reconstruction formula is identical to the scratch version, $A_k = U_k \Sigma_k V_k^T$, just built from TensorFlow's computed singular triplets, truncated to the top k . The variance retained calculation is also the same ratio as before, $\sum_{i=1}^k \sigma_i^2 / \sum_{\text{all } i} \sigma_i^2$, but here the denominator is computed directly as the sum of squares of every singular value TensorFlow returned, since the full spectrum is already available, no Frobenius-norm-of- A shortcut is needed the way it was in the scratch path.

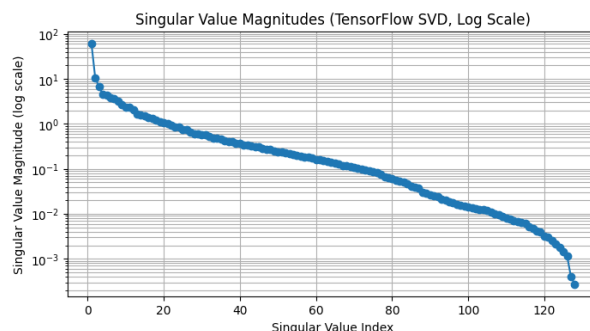
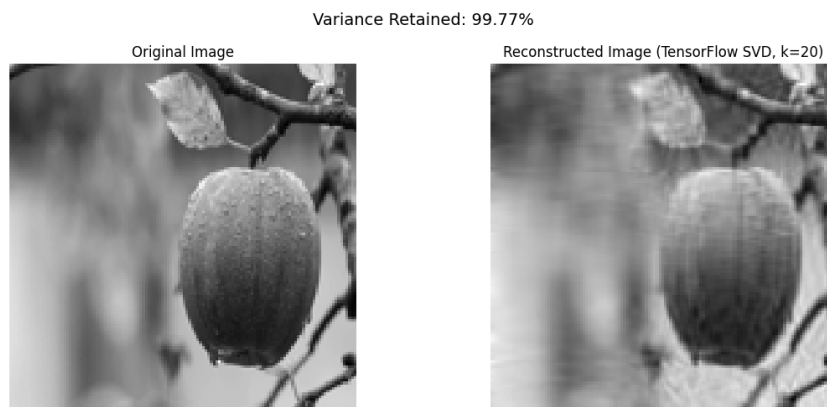
Output

```
TensorFlow image tensor shape: (128, 128)
Step 1: Performing SVD using tf.linalg.svd().
Shape of singular values (s): (128,)
Shape of U_tf: (128, 128)
Shape of V_tf (before transpose): (128, 128)
Shape of V_tf.T: (128, 128)
Top 20 Singular Values (from TensorFlow): [62.01269075 10.61735207  6.711951    4.5700015
 4.44934598]...

Step 2: Reconstructing image using top-k singular values/vectors.
Shape of reconstructed matrix: (128, 128)

Step 3: Computing reconstruction error (MSE) and % variance retained.
Mean Squared Error (MSE) for TensorFlow SVD reconstruction: 0.000579
% Variance Retained for TensorFlow SVD reconstruction: 99.77%

Step 4: Plotting results.
```



Actual VS Predicted Output

Inference

The scratch implementation and TensorFlow's `tf.linalg.svd` produce the same top-20 singular values to the displayed precision (62.013, 10.617, 6.712, 4.570, 4.449, ...), and consequently identical MSE (0.000579) and % variance retained (99.77%) at $k = 20$. This agreement is strong evidence that the power-iteration-with-deflation implementation is numerically correct, since it converges to the same eigenpairs of $A^T A$ that the library's internal routine finds directly.

The fact that just 20 of the 128 possible singular values retain 99.77% of the image's total energy shows that the image's information is highly concentrated in a small number of dominant directions, consistent with the usual argument for why SVD-based compression works well on natural images. This concentration likely reflects that the source image has large smooth/low-frequency regions rather than

fine texture or noise, since noisy or highly textured images spread energy more evenly across singular values and would need larger k for the same variance retained.

2 MLP: Scratch Code and Tensor Flow Implementation

Problem Statement

The goal of this section is to predict a student's final grade ($G3$, on a 0–20 scale) from five features: study time, number of past class failures, number of absences, and the two earlier-period grades $G1$ and $G2$. This is a continuous-valued regression problem with a small number of numeric, tabular features, no sequential or spatial structure, which makes a small feed-forward Multi-Layer Perceptron (MLP) a natural fit: a couple of nonlinear hidden layers are enough to capture any non-linear interaction between study habits/failures/absences and the two prior grades, while a linear output unit is appropriate since $G3$ is unbounded-in-principle and not a classification target. The section first builds this MLP entirely from scratch (manual forward pass, backpropagation, and gradient descent) to make the underlying mechanics explicit, then reimplements the same architecture in TensorFlow/Keras to check the scratch implementation against a standard library and to compare optimizers (plain gradient descent vs. Adam). The scratch implementation is available at `mlp.ipynb`.

From-Scratch Implementation

Dataset Preparation

```
features = ['studytime', 'failures', 'absences', 'G1', 'G2']
target = 'G3'
X = df[features].values
y = df[target].values.reshape(-1, 1)
```

`df[features]`, a list of five column names, returns a DataFrame, and `.values` converts it to a genuinely 2D array,

$$X \in \mathbb{R}^{N \times 5},$$

with each column holding one feature in the listed order: study time, number of past failures, absences, first-period grade $G1$, second-period grade $G2$. `df[target]`, a single string, returns a 1D Series, so `.values` gives shape $(N,)$; `reshape(-1,1)` forces it into a column vector $y \in \mathbb{R}^{N \times 1}$.

Train-Test Split and Standardization

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

x_scaler = StandardScaler()
X_train_scaled = x_scaler.fit_transform(X_train)
X_test_scaled = x_scaler.transform(X_test)

y_scaler = StandardScaler()
y_train_scaled = y_scaler.fit_transform(y_train)
y_test_scaled = y_scaler.transform(y_test)
```

The split reserves 20% of rows for testing, with the same row randomly permuted but consistently for X and y together, fixed by `random_state=42` for reproducibility. Standardization, separately for inputs and target, applies $z = (x - \mu)/\sigma$ column-wise, with μ, σ computed only on the training data via `fit_transform` and reused unchanged on the test data via `transform`. As before, this keeps the features and target on a comparable, centered scale, which matters even more here because this network's hidden layers use ReLU rather than tanh: ReLU has no upper saturation bound, so without scaling, large input magnitudes could produce very large pre-activations and correspondingly large, unstable gradients during the early stages of training.

Activation Functions

```
def relu(Z):
    return np.maximum(0, Z)

def relu_derivative(Z):
    return (Z > 0).astype(float)

def linear(Z):
    return Z
```


ReLU is defined as

$$\text{ReLU}(Z) = \max(0, Z), \quad \text{ReLU}'(Z) = \begin{cases} 1 & Z > 0 \\ 0 & Z \leq 0 \end{cases},$$

used for the hidden layers. Unlike tanh, ReLU does not saturate for large positive inputs, its gradient stays exactly 1 there rather than shrinking toward zero, which is part of why it largely avoids the vanishing-gradient behavior tanh-based networks like the earlier RNN can suffer from. The linear activation, $\text{linear}(Z) = Z$ with derivative 1 everywhere, is used only at the output layer, appropriate here because $G3$ is a continuous regression target with no natural bound the activation should enforce; squashing the final layer through ReLU or tanh would needlessly restrict the range of predictable values.

Loss Function

```
def mean_squared_error_loss(y_true, y_pred):
    return 0.5 * np.mean(np.square(y_pred - y_true))
```

$$\mathcal{L} = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2,$$

where m is the number of samples in the batch. This differs from the RNN's per-sample loss in one respect: here `np.mean` averages over all m training examples at once, since this network processes the entire training set in a single forward pass (full-batch), rather than looping sample by sample. The $\frac{1}{2}$ factor again exists purely so the derivative loses the 2 that differentiating a square produces.

Parameter Initialization

```
if l < L - 1:
    parameters[f'W{l}'] = np.random.randn(layer_sizes[l], layer_sizes[l-1]) * np.sqrt(2. / layer_sizes[l-1])
else:
    parameters[f'W{l}'] = np.random.randn(layer_sizes[l], layer_sizes[l-1]) * 0.01
parameters[f'b{l}'] = np.zeros((layer_sizes[l], 1))
```

Hidden layer weights use He initialization,

$$W^{[l]} \sim \mathcal{N}(0, 1) \cdot \sqrt{\frac{2}{n^{[l-1]}}, \quad n^{[l-1]} = \text{layer_sizes}[l-1],$$

the number of units feeding into that layer. The motivation is variance preservation: ReLU zeroes out roughly half its inputs on average, so a layer's output variance would shrink layer to layer without compensation; scaling weights by $\sqrt{2/n^{[l-1]}}$ is specifically tuned so the variance of the pre-activations $Z^{[l]}$ stays roughly constant from one layer to the next, given ReLU activations, preventing the signal from vanishing or exploding as it passes through several layers. The output layer instead uses a small fixed scale, $\times 0.01$, since it's a single linear layer producing one scalar, He's ReLU-specific scaling doesn't apply there, and a small initial scale simply keeps the very first predictions modest before training shapes the weights properly. Biases, as before, start at zero, safe because the random weight matrices already break symmetry between units in the same layer.

Forward Propagation

```
A = X_input.T
for l in range(1, L):
    Z = np.dot(W, A) + b
    A = linear(Z) if l == L - 1 else relu(Z)
    caches.append({'Z': Z, 'A': A})
```

The input is transposed to shape (features, samples), $A^{[0]} = X^T \in \mathbb{R}^{5 \times m}$, so that each column is one student's feature vector. For each layer $l = 1, \dots, L - 1$:

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]}, \quad A^{[l]} = \begin{cases} \text{ReLU}(Z^{[l]}) & l < L - 1 \\ Z^{[l]} & l = L - 1 \end{cases}.$$

Weight shapes follow the same (output dim, input dim) convention as before, $W^{[l]} \in \mathbb{R}^{n^{[l]} \times n^{[l-1]}}$, which is what makes $W^{[l]}A^{[l-1]}$ a valid matrix multiplication producing the correct next-layer dimensionality. Every $Z^{[l]}$ and $A^{[l]}$ is cached, exactly as the RNN stored `xs` and `hs`, since backpropagation needs them.

Backward Propagation: Output Layer

```
dZ = y_pred_scaled - y_true_reshaped
grads[f'dW{L-1}'] = (1 / m) * np.dot(dZ, A_prev_output_layer.T)
grads[f'db{L-1}'] = (1 / m) * np.sum(dZ, axis=1, keepdims=True)
```

Because the output activation is linear, $\partial A^{[L-1]} / \partial Z^{[L-1]} = 1$, so the gradient of the loss with respect to the output pre-activation reduces to

$$dZ^{[L-1]} = \hat{y} - y,$$

a $(1, m)$ matrix, one entry per sample, computed here without the $1/m$ factor baked in yet. The averaging over the m samples, the $1/m$ from $\mathcal{L} = \frac{1}{2m} \sum (\cdot)^2$, is instead deferred and applied once, directly when forming each parameter gradient:

$$dW^{[L-1]} = \frac{1}{m} dZ^{[L-1]} \left(A^{[L-2]} \right)^T, \quad db^{[L-1]} = \frac{1}{m} \sum_{i=1}^m dZ^{[L-1]}_{:,i}.$$

This is mathematically equivalent to scaling dZ by $1/m$ at the start, since that constant factor simply carries through unchanged across every subsequent matrix multiplication and elementwise operation in the chain rule, so applying it once at the very end, when the parameter gradients are finally assembled, gives the identical result with less repeated arithmetic.

Backward Propagation: Hidden Layers

```
for l_param_idx in reversed(range(1, L-1)):
    dZ = np.dot(parameters[f'W{l_param_idx + 1}'].T, dZ) * relu_derivative(caches[l_param_idx-1]['Z'])
    grads[f'dW{l_param_idx}'] = (1 / m) * np.dot(dZ, A_prev_hidden_layer.T)
    grads[f'db{l_param_idx}'] = (1 / m) * np.sum(dZ, axis=1, keepdims=True)
```

Walking backward through the hidden layers, the chain rule at each layer l is

$$dZ^{[l]} = \left(\left(W^{[l+1]} \right)^T dZ^{[l+1]} \right) \odot \text{ReLU}'(Z^{[l]}),$$

where $\odot$ is elementwise multiplication. $\left(W^{[l+1]} \right)^T dZ^{[l+1]}$ pushes the error from the next layer's pre-activation space back into the current layer's activation space, exactly analogous to $W_{hh}^T \delta_t$ in the RNN's backward pass, just using the next layer's weight matrix here instead of a recurrent weight reused across time. Multiplying by $\text{ReLU}'(Z^{[l]})$, which is 1 wherever $Z^{[l]} > 0$ and 0 otherwise, zeroes out the gradient at exactly the units that were inactive (clipped to zero) during the forward pass, since those units contributed nothing to the output and so receive no learning signal. Once $dZ^{[l]}$ is known, $dW^{[l]}$ and $db^{[l]}$ are computed the same way as the output layer, using whichever activation matrix fed into that layer, the original input X^T for the first hidden layer, or the previous hidden layer's cached activation otherwise.

Training Loop: Gradient Descent

```
for l in range(1, L):
    parameters[f'W{l}'] -= learning_rate * grads[f'dW{l}']
    parameters[f'b{l}'] -= learning_rate * grads[f'db{l}']
```

$$W^{[l]} \leftarrow W^{[l]} - \eta dW^{[l]}, \quad b^{[l]} \leftarrow b^{[l]} - \eta db^{[l]}.$$

This update runs once per epoch, on gradients already averaged across the entire training set in one full-batch forward and backward pass, in contrast to the RNN code, which updated weights once per individual sample. With $m^{[1]} = 10$, $m^{[2]} = 5$, $m^{[3]} = 1$ units (`layer_sizes = [5, 10, 5, 1]`), learning rate $\eta = 0.01$, training runs for 2000 such full-batch updates.

Model Evaluation

```
y_pred_test_scaled, _ = forward_propagation(X_test_scaled, parameters, layer_sizes)
y_pred_test = y_scaler.inverse_transform(y_pred_test_scaled.T)
y_pred_test = np.clip(y_pred_test, 0, 20)

mse = mean_squared_error(y_test_original, y_pred_test)
mae = mean_absolute_error(y_test_original, y_pred_test)
```

Only the forward pass runs on the held-out test set, no gradients. `inverse_transform` reverses standardization the same way as before, $x = z\sigma + \mu$, using the training set's stored statistics, converting predictions back to the original 0–20 grade scale. `np.clip(y_pred_test, 0, 20)` then forces any prediction that fell outside the valid grade range, which can happen since the linear output layer has no awareness that $G3$ is bounded, back into $[0, 20]$. Evaluation uses two metrics:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad \text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|,$$

both computed on the original, unscaled grade values; MAE is less sensitive to large individual errors than MSE, since it doesn't square the residuals, giving a complementary view of typical prediction error.

Visualization

```
plt.plot(range(1, len(costs) + 1), costs)
plt.scatter(y_test_original, y_pred_test, alpha=0.7)
plt.plot([0, 20], [0, 20], 'r--')
```

The first plot tracks $\mathcal{L}$ against epoch number, a standard convergence diagnostic, ideally decreasing and flattening as training progresses. The second plots each test student's actual grade against the model's predicted grade as a scatter point, with the dashed diagonal line $y = x$ marking where a perfect prediction would land, points clustering tightly around that line indicate good fit, while a wide scatter or systematic offset from it indicates where the model is over- or under-predicting.

Output

```
===== 1. Dataset Preparation =====
Original dataset shape: (395, 33)
Features (X) shape: (395, 5)
Target (y) shape: (395, 1)
First 5 rows of features:
  studytime  failures  absences  G1  G2
0         2         0         6   5   6
1         2         0         4   5   5
2         2         3        10   7   8
3         3         0         2  15  14
4         2         0         4   6  10
First 5 rows of target:
   G3
0   6
1   6
2  10
3  15
4  10

===== 2. Data Preprocessing =====
Training set size: 316 samples
Test set size: 79 samples
Features and target standardized.
X_train_scaled shape: (316, 5)
y_train_scaled shape: (316, 1)

===== 3. MLP From Scratch Implementation =====

===== 4. Model Training =====
Training MLP with architecture: [5, 10, 5, 1]
Learning Rate: 0.01, Epochs: 2000
```

```

Epoch 100/2000, Loss: 0.299118
Epoch 200/2000, Loss: 0.218970
Epoch 300/2000, Loss: 0.180268
Epoch 400/2000, Loss: 0.159790
Epoch 500/2000, Loss: 0.147616
Epoch 600/2000, Loss: 0.139400
Epoch 700/2000, Loss: 0.132427
Epoch 800/2000, Loss: 0.126471
Epoch 900/2000, Loss: 0.121369
Epoch 1000/2000, Loss: 0.117107
Epoch 1100/2000, Loss: 0.113389
Epoch 1200/2000, Loss: 0.110053
Epoch 1300/2000, Loss: 0.107134
Epoch 1400/2000, Loss: 0.104569
Epoch 1500/2000, Loss: 0.102330
Epoch 1600/2000, Loss: 0.100299
Epoch 1700/2000, Loss: 0.098438
Epoch 1800/2000, Loss: 0.096713
Epoch 1900/2000, Loss: 0.095117
Epoch 2000/2000, Loss: 0.093632

```

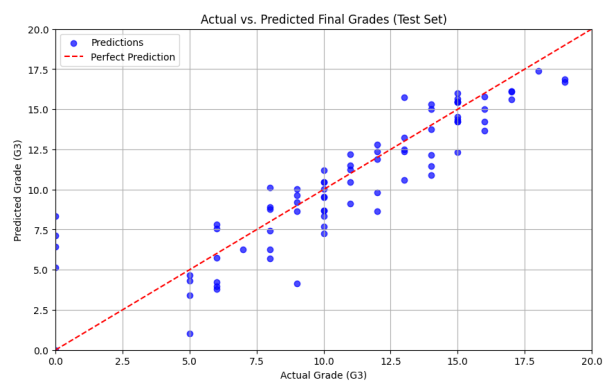
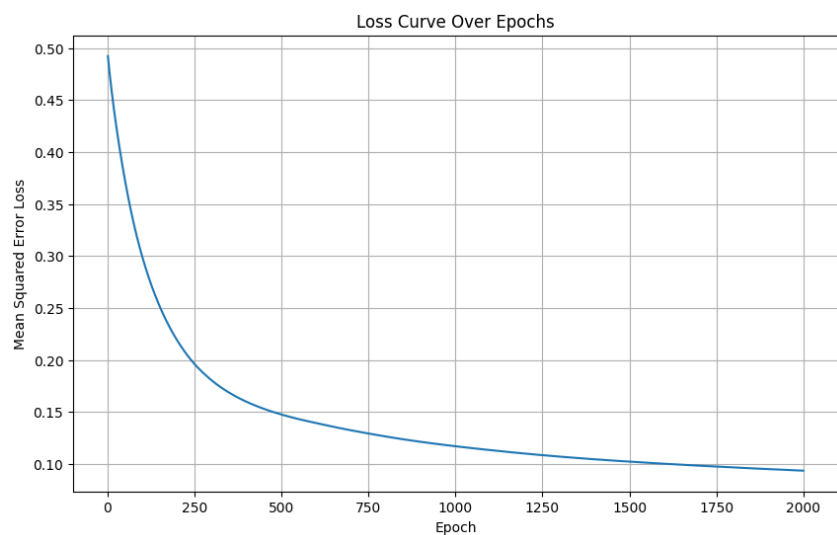
Training complete.

===== 5. Model Evaluation =====

Mean Squared Error (MSE) on test set (original grade scale): 4.8123

Mean Absolute Error (MAE) on test set (original grade scale): 1.5383

===== 6. Visualization =====



Actual VS Predicted Output

TensorFlow/Keras Implementation

Data loading, the train/test split, and the standardization of features and target are unchanged from the from-scratch implementation above and are reused directly (`X_train_scaled`, `y_train_scaled`, `X_test_scaled`, `y_scaler`, `y_test_original`), so only the model definition, compilation, training call, and evaluation are shown below.

Keras Equivalent Model

```
keras_model = keras.Sequential([
    layers.Input(shape=(input_dim,)),
    layers.Dense(10, activation='relu'),
    layers.Dense(5, activation='relu'),
    layers.Dense(output_dim, activation='linear')
])
keras_model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.01), loss='mse')
```

Each `Dense` layer internally implements the same $Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$ followed by an activation, matching the scratch architecture's two hidden layers, 10 then 5 ReLU units, and a single linear output unit. `loss='mse'` is Keras's default mean squared error, $\frac{1}{m} \sum (\hat{y} - y)^2$, without the $\frac{1}{2}$ factor used in the scratch loss; this constant factor of 2 difference doesn't change which direction gradient descent moves in, only its raw magnitude, so it doesn't change the optimization path. The optimizer here is Adam rather than plain gradient descent: Adam maintains a running estimate of each parameter's gradient mean (first moment, momentum-like) and squared-gradient mean (second moment, RMSProp-like), and uses both to adaptively rescale the effective step size per parameter, generally converging faster and more robustly than fixed-learning-rate gradient descent, which is what the scratch implementation uses.

Keras Training and Evaluation

```
history = keras_model.fit(X_train_scaled, y_train_scaled, epochs=num_epochs, batch_size=32, verbose=0)

y_pred_keras_scaled = keras_model.predict(X_test_scaled)
y_pred_keras = y_scaler.inverse_transform(y_pred_keras_scaled)
y_pred_keras = np.clip(y_pred_keras, 0, 20)
```

With `batch_size=32`, Keras performs mini-batch gradient descent, splitting the training set into chunks of 32 samples and updating weights once per chunk, several updates per epoch, rather than the scratch version's one full-batch update per epoch. Evaluation mirrors the scratch path exactly: forward pass on the test set, inverse transform back to the 0–20 grade scale, clipping to the valid range, then the same MSE and MAE formulas computed on the original scale, allowing a direct, apples-to-apples comparison between the scratch and Keras implementations' final printed metrics.

Output

```
===== Keras MLP Implementation =====
Model: "sequential"
```

Layer (type)	Output Shape	Param #
hidden_layer_1 (Dense)	(None, 10)	60
hidden_layer_2 (Dense)	(None, 5)	55
output_layer (Dense)	(None, 1)	6

```
Total params: 121 (484.00 B)
Trainable params: 121 (484.00 B)
Non-trainable params: 0 (0.00 B)
```

```
Training Keras MLP with architecture: [5, 10, 5, 1]
Learning Rate: 0.01, Epochs: 2000
```

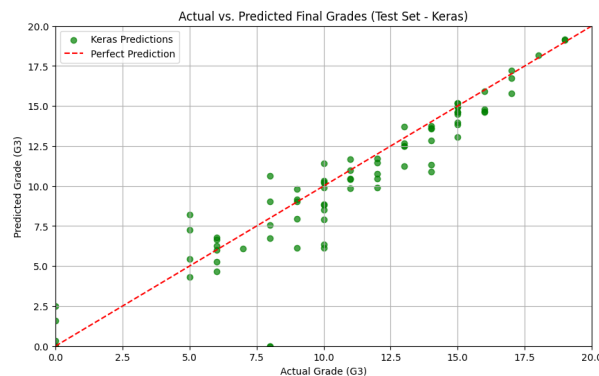
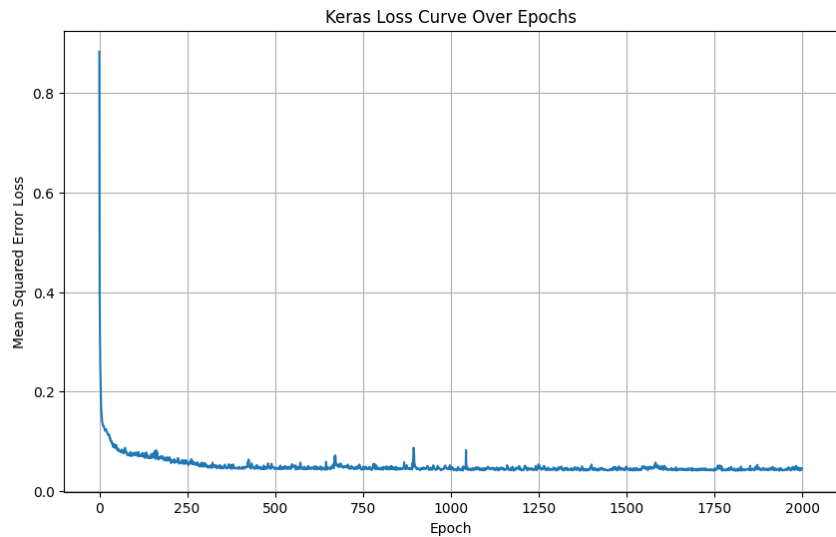
```
Keras Training complete.
```

```

===== Keras Model Evaluation =====
3/3 - 0s 26ms/step
Keras Mean Squared Error (MSE) on test set (original grade scale): 3.3431
Keras Mean Absolute Error (MAE) on test set (original grade scale): 1.1395

Scratch MSE: 4.8123, Keras MSE: 3.3431
Scratch MAE: 1.5383, Keras MAE: 1.1395

```



Actual VS Predicted Output

Inference

Both models were trained for the same 2000 epochs with the same $[5, 10, 5, 1]$ architecture and the same 0.01 learning rate, on the identical train/test split (316/79 students, from `student-mat.csv` only). The Keras/Adam version reached a lower test-set error than the scratch/plain-gradient-descent version on both metrics: MSE dropped from 4.8123 to 3.3431, and MAE dropped from 1.5383 to 1.1395 grade points (on a 0–20 scale). In relative terms, Keras's MSE is roughly 30% lower and its MAE roughly 26% lower than the scratch model's.

The gap between the two models is more likely attributable to *how* they were trained than to the architecture itself, since the architecture is identical in both:

- **Optimizer:** Adam adapts its per-parameter step size using running gradient-moment estimates, whereas the scratch version uses plain fixed-learning-rate gradient descent, which typically converges slower and can get stuck with a suboptimal fixed step size.
- **Update frequency:** `batch_size=32` on 316 training samples gives about 10 weight updates per epoch, versus 1 full-batch update per epoch in the scratch version.

3 RNN: Scratch Code and TensorFlow Implementation

Problem Statement

The goal is to predict the next day's temperature from a short sequence of same-day readings. Each sample in `temp_vanilla.csv` consists of three temperature readings taken at fixed points in a single day — morning, afternoon, and evening — and a target label, the temperature recorded the following day. The three readings are not independent features but an ordered progression through the day, where later readings are influenced by earlier ones (e.g. a hot morning tends to precede a hot afternoon). A Recurrent Neural Network (RNN) is a natural fit here rather than a standard feedforward network which treats the three readings as an unordered feature vector.

Two implementations are compared: a scratch NumPy implementation, where the forward pass, backpropagation through time (BPTT), and gradient descent update are all written out explicitly to expose the underlying mechanics, and a TensorFlow/Keras implementation using a built-in `SimpleRNN` layer, which performs the same computation but with the recurrence and gradient computation handled internally by the framework.

The full scratch implementation is available at `rnn.ipynb`.

Scratch Code

Loading the Data: Column Vectors and Reshaping

```
X = df[["temp_morning", "temp_afternoon", "temp_evening"]].values
y = df["next_day_temp"].values.reshape(-1, 1)
```

Selecting multiple columns with double brackets, `df[["..."]]`, returns a DataFrame, and `.values` on it gives a genuinely 2D array. So

$$X \in \mathbb{R}^{100 \times 3}, \quad \text{column } j \text{ of } X = \text{feature } j \text{ (morning, afternoon, evening, in that order).}$$

Selecting a single column with single brackets, `df["next_day_temp"]`, returns a 1D Series, and `.values` gives shape `(100,)`, no column dimension. `reshape(-1,1)` forces this into a genuine column vector:

$$y \in \mathbb{R}^{100 \times 1}.$$

The `-1` tells NumPy to infer that dimension automatically from the total element count, here $100 \times 1 = 100$, so it resolves to 100 rows. Equivalently, `df[["next_day_temp"]].values` (double brackets) gives the same `(100,1)` shape directly, since a one-column DataFrame is already 2D. This 2D shape matters later: NumPy treats `(100,)` and `(100,1)` differently under matrix multiplication and broadcasting.

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42)
```

A single random permutation of row indices is generated and applied identically to X and y , preserving the pairing between each day's readings and its target. With `test_size=0.2`, 20% of rows are held out:

$$X_{\text{train}} \in \mathbb{R}^{80 \times 3}, X_{\text{test}} \in \mathbb{R}^{20 \times 3}, y_{\text{train}} \in \mathbb{R}^{80 \times 1}, y_{\text{test}} \in \mathbb{R}^{20 \times 1}.$$

`random_state=42` fixes the seed of the underlying pseudo-random generator, making the split reproducible across runs.

Standard Scaling

```
X_train = x_scaler.fit_transform(X_train)
X_test = x_scaler.transform(X_test)
y_train = y_scaler.fit_transform(y_train)
y_test_scaled = y_scaler.transform(y_test)
```

Each feature column is standardized, per column, as

$$z = \frac{x - \mu}{\sigma},$$

with μ, σ computed only from the training data (`fit_transform`), then reused on the test data (`transform`, no re-fitting), preventing test statistics from leaking into preprocessing. `x_scaler` and `y_scaler` are kept separate since X and y have different means and spreads, and `y_scaler` alone is needed later to invert predictions correctly.

Why scaling matters: unscaled features distort the loss surface, inflating the condition number of the Hessian (ratio of its largest to smallest eigenvalue), turning loss contours into narrow ellipses that a single fixed learning rate navigates poorly. It also keeps pre-activation values entering $\tanh$ close to its near-linear region around zero, avoiding early saturation, where $\tanh' = 1 - \tanh^2 \approx 0$ and gradients stop flowing. It is also consistent with the small weight initialization scale used below.

RNN Hyperparameters

```
input_size = 1    # d = 1
hidden_size = 8   # h = 8
output_size = 1   # k = 1
T = 3
learning_rate = 0.001
epochs = 1000
```

$d = 1$: at each timestep the network sees one scalar reading, not all three temperatures at once — this is what makes it a sequence model. $h = 8$: dimension of the hidden state, the network’s memory. $k = 1$: dimension of the output. $T = 3$: number of recurrence steps (morning, afternoon, evening). $\eta = 0.001$: gradient descent step size. 1000 epochs: full passes over all 80 training sequences.

Because the same weights (W_{xh}, W_{hh}, b_h) are reused at every timestep and their shapes depend only on d and h , never on T , the same architecture could process sequences of any length by simply changing how many times the loop runs — RNNs are naturally suited to variable-length sequences, though here T is fixed at 3 for every sample.

Weight Initialization

```
Wxh = np.random.randn(hidden_size, input_size) * 0.1
Whh = np.random.randn(hidden_size, hidden_size) * 0.1
bh = np.zeros((hidden_size, 1))
Why = np.random.randn(output_size, hidden_size) * 0.1
by = np.zeros((output_size, 1))
```

$$W_{xh} \in \mathbb{R}^{h \times d} = \mathbb{R}^{8 \times 1}, \quad W_{hh} \in \mathbb{R}^{h \times h} = \mathbb{R}^{8 \times 8}, \quad W_{hy} \in \mathbb{R}^{k \times h} = \mathbb{R}^{1 \times 8}.$$

Shapes follow the (output dim, input dim) convention, which is exactly what makes left-multiplication, e.g. $W_{xh}x_t$, produce the correct output dimensionality. Weights are drawn from $\mathcal{N}(0, 1)$ then scaled by 0.1 to keep pre-activations small (avoiding $\tanh$ saturation). Biases b_h, b_y start at zero — safe because the random weight matrices already break symmetry between hidden units, so biases don’t need randomness to differentiate them.

Forward Pass

```
h_prev = np.zeros((hidden_size, 1))
for t in range(T):
    x_t = np.array([[X_train[n, t]]])
    h_prev = np.tanh(Wxh @ x_t + Whh @ h_prev + bh)
y_hat = Why @ hs[T-1] + by
```

Hidden state starts at $h_0 = \mathbf{0}$. For $t = 1, \dots, T$:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h).$$

The new hidden state blends the current input (via W_{xh}) and the previous hidden state (via W_{hh}), squashed by $\tanh$ into $(-1, 1)$. After T steps, h_T is a compressed summary of the whole sequence, read out linearly:

$$\hat{y} = W_{hy}h_T + b_y.$$

Loss Function

```
loss = 0.5*np.sum((y_hat-target)**2)
```

$$\mathcal{L} = \frac{1}{2} (\hat{y} - y)^2.$$

The factor $\frac{1}{2}$ cancels the 2 produced by differentiating the square, giving a clean derivative.

Output Layer Gradient

```
dy = y_hat - target
dWhy = dy @ hs[T-1].T
dby = dy
dh_next = Why.T @ dy
```

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = \delta_y, \quad \frac{\partial \mathcal{L}}{\partial W_{hy}} = \delta_y h_T^\top, \quad \frac{\partial \mathcal{L}}{\partial b_y} = \delta_y.$$

Pushing the error back into hidden space (chain rule through $\hat{y} = W_{hy}h_T + b_y$):

$$\frac{\partial \mathcal{L}}{\partial h_T} = W_{hy}^\top \delta_y.$$

Backpropagation Through Time (BPTT)

```
for t in reversed(range(3)):
    h = hs[t]
    da = dh_next * (1 - h**2)
    dbh += da
    dWxh += da @ xs[t].T
    if t > 0:
        dWhh += da @ hs[t-1].T
    dh_next = Whh.T @ da
```

For $t = T, \dots, 1$, using $\tanh'(z) = 1 - \tanh^2(z) = 1 - h_t^2$:

$$\delta_t = \frac{\partial \mathcal{L}}{\partial h_t} \odot (1 - h_t^2)$$

where $\odot$ is elementwise multiplication. Since W_{xh}, W_{hh}, b_h are each reused at every timestep, their gradients are sums across all timesteps they touched:

$$\frac{\partial \mathcal{L}}{\partial b_h} = \sum_{t=1}^T \delta_t, \quad \frac{\partial \mathcal{L}}{\partial W_{xh}} = \sum_{t=1}^T \delta_t x_t^\top, \quad \frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=2}^T \delta_t h_{t-1}^\top.$$

The error is carried one step further back via

$$\frac{\partial \mathcal{L}}{\partial h_{t-1}} = W_{hh}^\top \delta_t.$$

This repeated multiplication by $W_{hh}^\top$ at every backward step is exactly the mechanism behind vanishing/exploding gradients in long sequences — not an issue here with $T = 3$, but the same root cause as in LSTMs, which is why LSTMs add gating to control it.

Weight Update (Gradient Descent)

```
Wxh -= learning_rate*dWxh
Whh -= learning_rate*dWhh
bh -= learning_rate*dbh
Why -= learning_rate*dWhy
by -= learning_rate*dby
```

$$W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial W}, \quad \text{for every } W \in \{W_{xh}, W_{hh}, b_h, W_{hy}, b_y\}.$$

This update happens once per training sample (not per epoch), so with 80 samples this is effectively stochastic gradient descent, repeated for 1000 epochs.

Testing — Forward Pass Only

```
for n in range(len(X_test)):
    h_prev = np.zeros((hidden_size,1))
    for t in range(3):
        x_t = np.array([[X_test[n,t]]])
        h_prev = np.tanh(Wxh @ x_t + Whh @ h_prev + bh)
    y_hat = Why @ h_prev + by
```

Identical recurrence as training, but with the now-trained weights and no gradient computation, applied to the 20 held-out sequences.

Inverse Transform

```
predictions = y_scaler.inverse_transform(predictions)
```

Reverses the earlier standardization using the training set's stored μ, σ :

$$x = z \cdot \sigma + \mu,$$

converting predictions from standardized scale back into real temperature units.

Evaluation

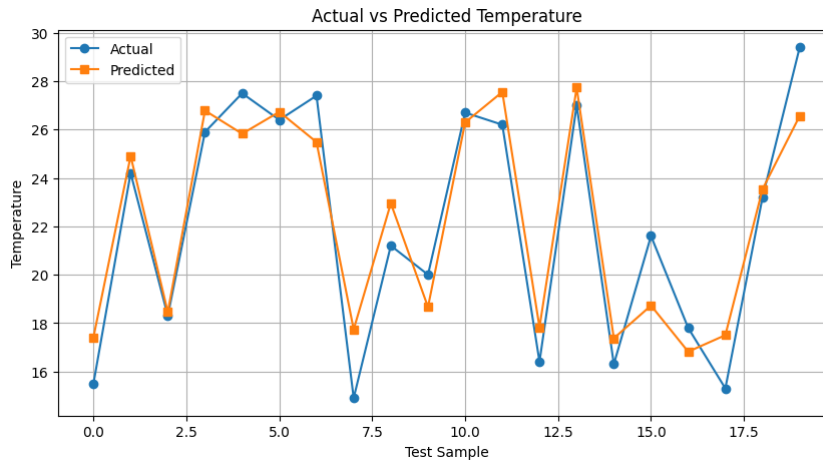
```
mse = mean_squared_error(actual, predictions)
```

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i^{\text{actual}} - \hat{y}_i)^2, \quad N = 20,$$

computed in real temperature units, summarizing average prediction error on the unseen test days.

Output

```
(80, 3)
(80, 1)
Epoch   0  Loss = 0.507923
Epoch 100  Loss = 0.045324
Epoch 200  Loss = 0.045298
Epoch 300  Loss = 0.045278
Epoch 400  Loss = 0.045259
Epoch 500  Loss = 0.045242
Epoch 600  Loss = 0.045225
Epoch 700  Loss = 0.045209
Epoch 800  Loss = 0.045192
Epoch 900  Loss = 0.045175
MSE : 2.6197
```



Actual VS Predicted Output

TensorFlow Version

```

model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(
        hidden_size,
        activation="tanh",
        input_shape=(T, 1)
    ),
    tf.keras.layers.Dense(1)
])

optimizer = tf.keras.optimizers.SGD(learning_rate=learning_rate)

# loss = 0.5 * sum of squared error
def half_sse(y_true, y_pred):
    return 0.5 * tf.reduce_sum(tf.square(y_true - y_pred))

model.compile(optimizer=optimizer, loss=half_sse)

history = model.fit(
    X_train_seq,
    y_train,
    epochs=epochs,
    batch_size=1,
    verbose=0
)

```

Data loading, the train-test split, and standard scaling are identical to the scratch implementation above — same columns, same 80/20 split with `random_state=42`, same per-column $z = (x - \mu)/\sigma$ standardization fit only on training data. The only structural difference is the reshape into `(samples, timesteps, features) = (-1, 3, 1)`, which is TensorFlow's required input format for a recurrent layer, matching the same sequence-of-3-scalars view used in the scratch forward pass.

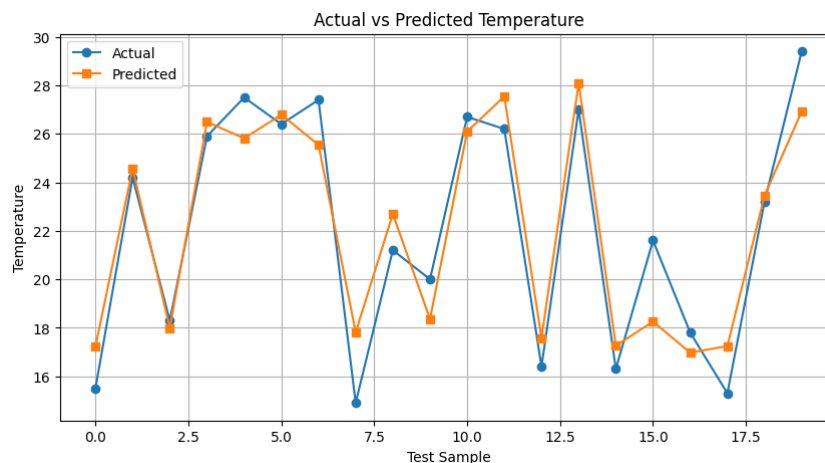
`SimpleRNN(hidden_size, activation="tanh")` is TensorFlow's built-in version of the same recurrence derived by hand above, $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$, internally owning W_{xh}, W_{hh}, b_h with identical shapes and looping over $T = 3$ timesteps, but hidden inside the layer. `Dense(1)` is the readout layer, equivalent to $\hat{y} = W_{hy}h_T + b_y$, with W_{hy}, b_y created automatically.

Keras's built-in losses default to *mean* squared error rather than the $\frac{1}{2}(\hat{y} - y)^2$ used in the scratch derivation, so `half_sse` is defined to match it exactly, keeping the two implementations loss-for-loss comparable. `SGD(learning_rate=0.001)` matches the plain gradient descent update rule $W \leftarrow W - \eta \partial \mathcal{L} / \partial W$ used above, with no momentum or adaptive learning rate.

`model.fit(..., batch_size=1)` replaces the entire manual forward/backward/update loop written out step-by-step above: one sample is forward-propagated, its loss computed, gradients backpropagated through all 3 timesteps via TensorFlow's automatic differentiation (the same BPTT derived by hand), and weights updated — once per sample, per epoch. Testing, inverse transform, MSE, and plotting are unchanged from the scratch version's logic.

Output

```
(80, 3)
(80, 1)
Epoch   0  Loss = 0.486807
Epoch 100  Loss = 0.043121
Epoch 200  Loss = 0.042752
Epoch 300  Loss = 0.042487
Epoch 400  Loss = 0.042284
Epoch 500  Loss = 0.042122
Epoch 600  Loss = 0.041993
Epoch 700  Loss = 0.041890
Epoch 800  Loss = 0.041805
Epoch 900  Loss = 0.041736
MSE : 2.5496
```



Actual VS Predicted Output

Inference

Both implementations converge to a similar loss plateau and produce comparable test-set performance — MSE of 2.6197 for the scratch implementation versus 2.5496 for TensorFlow — confirming that the manually derived forward pass, BPTT, and gradient update logic match what TensorFlow’s `SimpleRNN` layer computes internally; the small gap in MSE is attributable to differences in weight initialization and minor numerical differences. The results suggest that same-day temperature readings carry a modest but real predictive signal for the next day’s temperature, though the residual error ($\text{MSE} \approx 2.5\text{--}2.6$, i.e. roughly $\pm 1.6^\circ$ average deviation) indicates that a 3-point daily sequence alone captures only part of the variability — additional context such as historical multi-day trends, seasonality, or weather station data would likely be needed to improve accuracy further.

4 LSTM: Scratch Code and Tensor Flow Implementation

Problem Statement

Stock closing prices form a sequential, temporally dependent signal: tomorrow's price is influenced by a window of recent past prices, and simple feedforward models can't naturally capture that ordering. An LSTM is suited to this because its gated recurrence (forget/input/output gates plus a cell state) lets it retain or discard information across a sequence, addressing the vanishing-gradient problem that plagues vanilla RNNs over longer dependencies. This section frames next-day closing price prediction as a sliding-window regression problem: given the last $T = 5$ closing prices, predict the next one. It first builds the LSTM forward/backward pass (BPTT) from scratch with NumPy to expose the gate mechanics and gradient flow explicitly, then reproduces the same architecture with a Keras LSTM layer to check that the scratch implementation's predictive performance (MSE) is in the same range as the library version. The full scratch implementation is available at [lstm.ipynb](#).

Scratch Code

Loading Data and Building Sliding Windows

```
prices = df["Close"].values

T = 5
X_list, y_list = [], []
for i in range(T, len(prices)):
    X_list.append(prices[i - T:i])
    y_list.append(prices[i])

X = np.array(X_list)
y = np.array(y_list).reshape(-1, 1)
```

`prices` is a 1D array of N_{raw} closing prices in ascending date order. The sliding window loop extracts every contiguous block of $T = 5$ consecutive prices as an input sequence, with the immediately following price as the target:

$$X^{(i)} = (p_{i-T}, p_{i-T+1}, \dots, p_{i-1}) \in \mathbb{R}^T, \quad y^{(i)} = p_i \in \mathbb{R}, \quad i = T, T+1, \dots, N_{\text{raw}} - 1.$$

This yields $N = N_{\text{raw}} - T$ samples. Stacking gives

$$X \in \mathbb{R}^{N \times T}, \quad y \in \mathbb{R}^{N \times 1}.$$

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

A single random permutation of row indices is applied identically to X and y , preserving each sequence-target pair. With `test_size=0.2`:

$$X_{\text{train}} \in \mathbb{R}^{[0.8N] \times T}, \quad X_{\text{test}} \in \mathbb{R}^{[0.2N] \times T}, \quad y_{\text{train}} \in \mathbb{R}^{[0.8N] \times 1}, \quad y_{\text{test}} \in \mathbb{R}^{[0.2N] \times 1}.$$

Note that `random_state=42` fixes the split reproducibly, but because the split is random rather than chronological, some test sequences may precede some training sequences in time — a data-leakage caveat for real forecasting use.

Standard Scaling

```
X_train = x_scaler.fit_transform(X_train)
X_test = x_scaler.transform(X_test)
y_train = y_scaler.fit_transform(y_train)
```

Each column of X (one lag position) and the scalar target y are standardized as

$$z = \frac{x - \mu}{\sigma},$$

with μ, σ estimated only from the training split (`fit_transform`), then reused on the test split (`transform`). Separate scalers x_scaler and y_scaler are kept because X and y live on different scales; y_scaler alone is needed later to invert predictions back to price units.

LSTM Hyperparameters

```
input_size = 1    # d = 1
hidden_size = 8   # h = 8
output_size = 1   # k = 1
# T = 5 (defined above)
learning_rate = 0.001
epochs = 1000
```

$d = 1$: at each timestep the network receives one scalar (a single lagged price), making the input at step t the vector $x_t \in \mathbb{R}^{d \times 1} = \mathbb{R}^{1 \times 1}$. $h = 8$: dimension of both the hidden state h_t and cell state c_t . $k = 1$: scalar output. $\eta = 0.001$: gradient descent step size. Updates happen once per training sample, so training is online (stochastic) SGD repeated for 1000 epochs.

Weight Initialization

```
Wxf = np.random.randn(hidden_size, input_size) * 0.1
Whf = np.random.randn(hidden_size, hidden_size) * 0.1
bf = np.zeros((hidden_size, 1))
# ... (same pattern for i, o, g gates)
Why = np.random.randn(output_size, hidden_size) * 0.1
by = np.zeros((output_size, 1))
```

Each gate has two weight matrices and a bias:

$$W_{x,\cdot} \in \mathbb{R}^{h \times d} = \mathbb{R}^{8 \times 1}, \quad W_{h,\cdot} \in \mathbb{R}^{h \times h} = \mathbb{R}^{8 \times 8}, \quad b_{\cdot} \in \mathbb{R}^{h \times 1},$$

giving four such triples for the forget (f), input (i), output (o), and candidate (g) gates, plus the readout pair $W_{hy} \in \mathbb{R}^{k \times h} = \mathbb{R}^{1 \times 8}$, $b_y \in \mathbb{R}^{k \times 1}$. All weight entries are drawn from $\mathcal{N}(0, 1)$ and scaled by 0.1; biases are zero-initialized.

Forward Pass — LSTM Gates and States

```
for t in range(T):
    x_t = np.array([[X_train[n, t]]])

    f_t = sigmoid(Wxf @ x_t + Whf @ hs[-1] + bf)
    i_t = sigmoid(Wxi @ x_t + Whi @ hs[-1] + bi)
    o_t = sigmoid(Wxo @ x_t + Who @ hs[-1] + bo)
    g_t = np.tanh(Wxg @ x_t + Whg @ hs[-1] + bg)

    c_t = f_t * cs[-1] + i_t * g_t
    h_t = o_t * np.tanh(c_t)
```

At each step $t = 1, \dots, T$, with $x_t \in \mathbb{R}^{d \times 1}$, $h_{t-1}, c_{t-1} \in \mathbb{R}^{h \times 1}$, the four gates are:

$$f_t = \sigma(W_{xf} x_t + W_{hf} h_{t-1} + b_f), \quad i_t = \sigma(W_{xi} x_t + W_{hi} h_{t-1} + b_i),$$

$$o_t = \sigma(W_{xo} x_t + W_{ho} h_{t-1} + b_o), \quad g_t = \tanh(W_{xg} x_t + W_{hg} h_{t-1} + b_g),$$

where $\sigma(z) = (1 + e^{-z})^{-1}$ and all operations are elementwise for gates. The cell and hidden states update as:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \quad h_t = o_t \odot \tanh(c_t),$$

where $\odot$ denotes elementwise multiplication. f_t scales how much of c_{t-1} to retain; $i_t \odot g_t$ is the new information written in; o_t gates what portion of c_t is exposed as h_t .

Forward Pass — Output and Loss

```
y_hat = Why @ hs[T] + by
target = np.array([[y_train[n, 0]])]
loss = 0.5 * np.sum((y_hat - target) ** 2)
```

After T recurrent steps, h_T is read out linearly:

$$\hat{y} = W_{hy} h_T + b_y \in \mathbb{R}^{k \times 1}.$$

The per-sample loss is

$$\mathcal{L} = \frac{1}{2} (\hat{y} - y)^2,$$

with $\frac{1}{2}$ introduced so the derivative is $\hat{y} - y$ with no leading constant.

Output Layer Gradient

```
dy = y_hat - target
dWhy = dy @ hs[T].T
dby = dy
dh_next = Why.T @ dy
```

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \hat{y} - y = \delta_y, \quad \frac{\partial \mathcal{L}}{\partial W_{hy}} = \delta_y h_T^\top, \quad \frac{\partial \mathcal{L}}{\partial b_y} = \delta_y.$$

The error is pushed back into hidden space via the chain rule through $\hat{y} = W_{hy} h_T + b_y$:

$$\frac{\partial \mathcal{L}}{\partial h_T} = W_{hy}^\top \delta_y.$$

BPTT — Gate Gradients

```
for t in reversed(range(T)):
    tanh_c = np.tanh(c_t)
    dh = dh_next

    do = dh * tanh_c
    dc = dh * o_t * (1 - tanh_c ** 2) + dc_next

    df = dc * c_prev
    di = dc * g_t
    dg = dc * i_t

    da_f = df * f_t * (1 - f_t)
    da_i = di * i_t * (1 - i_t)
    da_o = do * o_t * (1 - o_t)
    da_g = dg * (1 - g_t ** 2)
```

Working backwards through timestep t , let $\delta_h^{(t)} = \partial \mathcal{L} / \partial h_t$ (the sum of gradient flowing from $t+1$ and, at $t = T$, from the output). The cell gradient accumulates from both the hidden path and the future cell:

$$\frac{\partial \mathcal{L}}{\partial c_t} = \delta_h^{(t)} \odot o_t \odot (1 - \tanh^2(c_t)) + \delta_c^{(t+1)} \odot f_{t+1},$$

where $\delta_c^{(t+1)}$ carries the cell gradient from the step ahead (zero at $t = T$). Pre-activation gradients (i.e. before each gate's nonlinearity) are:

$$\begin{aligned} \Delta f_t &= \frac{\partial \mathcal{L}}{\partial c_t} \odot c_{t-1} \odot f_t \odot (1 - f_t), & \Delta i_t &= \frac{\partial \mathcal{L}}{\partial c_t} \odot g_t \odot i_t \odot (1 - i_t), \\ \Delta o_t &= \delta_h^{(t)} \odot \tanh(c_t) \odot o_t \odot (1 - o_t), & \Delta g_t &= \frac{\partial \mathcal{L}}{\partial c_t} \odot i_t \odot (1 - g_t^2), \end{aligned}$$

using $\sigma' = \sigma(1 - \sigma)$ and $\tanh' = 1 - \tanh^2$ evaluated at each gate's stored activation.

BPTT — Weight Gradients and State Propagation

```
dWxf += da_f @ xs[t].T; dWhf += da_f @ h_prev.T; dbf += da_f
dWxi += da_i @ xs[t].T; dWhi += da_i @ h_prev.T; dbi += da_i
dWxo += da_o @ xs[t].T; dWho += da_o @ h_prev.T; dbo += da_o
dWxg += da_g @ xs[t].T; dWhg += da_g @ h_prev.T; dbg += da_g

dh_next = Whf.T @ da_f + Whi.T @ da_i + Who.T @ da_o + Whg.T @ da_g
dc_next = dc * f_t
```

Since every weight matrix is reused at every timestep, its gradient accumulates as a sum across all T steps. For gate $\bullet \in \{f, i, o, g\}$:

$$\frac{\partial \mathcal{L}}{\partial W_{x,\bullet}} = \sum_{t=1}^T \Delta_{\bullet_t} x_t^\top, \quad \frac{\partial \mathcal{L}}{\partial W_{h,\bullet}} = \sum_{t=1}^T \Delta_{\bullet_t} h_{t-1}^\top, \quad \frac{\partial \mathcal{L}}{\partial b_\bullet} = \sum_{t=1}^T \Delta_{\bullet_t}.$$

The hidden gradient carried one step further back is:

$$\delta_h^{(t-1)} = W_{hf}^\top \Delta f_t + W_{hi}^\top \Delta i_t + W_{ho}^\top \Delta o_t + W_{hg}^\top \Delta g_t,$$

and the cell gradient carried back is:

$$\delta_c^{(t)} = \frac{\partial \mathcal{L}}{\partial c_t} \odot f_t.$$

Weight Update

```
Wxf -= learning_rate * dWxf; Whf -= learning_rate * dWhf; bbf -= learning_rate * dbf
Wxi -= learning_rate * dWxi; Whi -= learning_rate * dWhi; bi -= learning_rate * dbi
Wxo -= learning_rate * dWxo; Who -= learning_rate * dWho; bo -= learning_rate * dbo
Wxg -= learning_rate * dWxg; Whg -= learning_rate * dWhg; bg -= learning_rate * dbg
Why -= learning_rate * dWhy; by -= learning_rate * dby
```

$$W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial W}, \quad \text{for every } W \in \{W_{xf}, W_{hf}, b_f, W_{xi}, W_{hi}, b_i, W_{xo}, W_{ho}, b_o, W_{xg}, W_{hg}, b_g, W_{hy}, b_y\}.$$

The update is applied once per training sample (online SGD), so with $\lfloor 0.8N \rfloor$ samples per epoch and 1000 epochs there are $1000 \times \lfloor 0.8N \rfloor$ total gradient steps.

Testing — Forward Pass Only

```
for n in range(len(X_test)):
    h_prev = np.zeros((hidden_size, 1))
    c_prev = np.zeros((hidden_size, 1))
    for t in range(T):
        x_t = np.array([[X_test[n, t]]])
        f_t = sigmoid(Wxf @ x_t + Whf @ h_prev + bbf)
        i_t = sigmoid(Wxi @ x_t + Whi @ h_prev + bi)
        o_t = sigmoid(Wxo @ x_t + Who @ h_prev + bo)
        g_t = np.tanh(Wxg @ x_t + Whg @ h_prev + bg)
        c_prev = f_t * c_prev + i_t * g_t
        h_prev = o_t * np.tanh(c_prev)
    y_hat = Why @ h_prev + by
```

Identical recurrence to training — same gate equations, same readout $\hat{y} = W_{hy}h_T + b_y$ — but with fixed trained weights and no gradient computation, applied to each of the $\lfloor 0.2N \rfloor$ held-out sequences.

Inverse Transform and Evaluation

```
predictions = y_scaler.inverse_transform(predictions)
mse = mean_squared_error(actual, predictions)
```

Predictions are returned to price units by reversing the earlier standardization using the training set's stored μ, σ :

$$\hat{p} = \hat{z} \cdot \sigma + \mu.$$

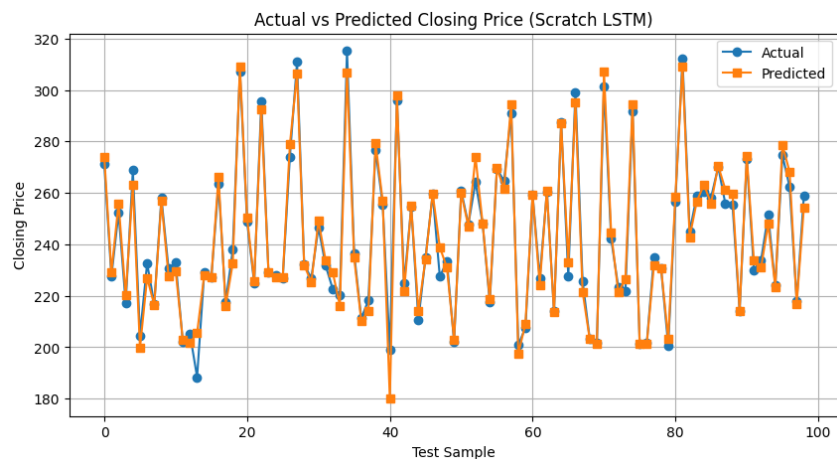
Performance is then summarized as

$$\text{MSE} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} (p_i - \hat{p}_i)^2,$$

computed in original price units.

Output

```
(396, 5)
(396, 1)
Epoch    0   Loss = 0.498937
Epoch   100   Loss = 0.021774
Epoch   200   Loss = 0.016704
Epoch   300   Loss = 0.014496
Epoch   400   Loss = 0.013157
Epoch   500   Loss = 0.012244
Epoch   600   Loss = 0.011589
Epoch   700   Loss = 0.011112
Epoch   800   Loss = 0.010766
Epoch   900   Loss = 0.010517
MSE : 17.3962
```



Actual VS Predicted Output

TensorFlow Version

Uses the same sliding-window data (`X_train`, `X_test`, `y_train`, `y_test`), `StandardScaler` objects, and `T/input_size/hidden_size/output_size/learning_rate/epochs` settings as the scratch version above; no data loading or scaling is repeated here. The only differences are the model definition, compilation, and training call.

```
X_train_keras = X_train.reshape(X_train.shape[0], T, input_size)
X_test_keras = X_test.reshape(X_test.shape[0], T, input_size)

model = Sequential([
    LSTM(hidden_size, activation='tanh', recurrent_activation='sigmoid', input_shape=(T, input_size)),
    Dense(output_size, activation='linear')
])

optimizer = Adam(learning_rate=learning_rate)
model.compile(optimizer=optimizer, loss='mse')

model.fit(
    X_train_keras,
    y_train,
    epochs=epochs,
    batch_size=len(X_train_keras),
    verbose=0
)
```

The reshape converts each sample from a flat length- T vector into the (timesteps, features) = $(T, 1)$ shape Keras's LSTM layer expects. The `LSTM(hidden_size, ...)` layer implements the same four-gate recurrence (f, i, o, g gates, cell state c_t , hidden state h_t) as the scratch code, with `tanh` as the state activation and `sigmoid` as the gate activation, matching the scratch implementation's

choice of nonlinearities. `Dense(output_size, activation='linear')` is the same linear readout as $\hat{y} = W_{hy}h_T + b_y$. Using Adam instead of plain SGD is a difference from the scratch version, which used vanilla gradient descent; Adam adapts the learning rate per parameter using running estimates of the gradient's first and second moments, which can converge faster or differently than fixed-step SGD. `batch_size=len(X_train_keras)` runs full-batch gradient descent (all training samples per update) rather than the scratch code's per-sample online updates, so the two are not doing identical optimization trajectories even though they train for the same number of epochs.

```
keras_predictions_scaled = model.predict(X_test_keras)
keras_predictions = y_scaler.inverse_transform(keras_predictions_scaled)
keras_actual = y_test
kears_mse = mean_squared_error(keras_actual, keras_predictions)
```

Same inverse-transform-then-evaluate pattern as the scratch version, using the same `y_scaler` fitted earlier.

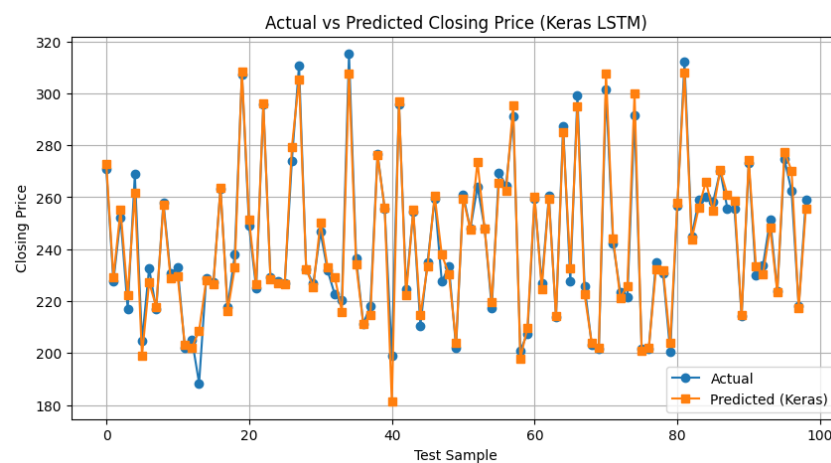
Output

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an
'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(
shape)' object as the first layer in the model instead.
super().__init__(**kwargs)
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
lstm_2 (LSTM)	(None, 8)	320
dense_2 (Dense)	(None, 1)	9

Actual VS Predicted Output

```
Total params: 329 (1.29 KB)
Trainable params: 329 (1.29 KB)
Non-trainable params: 0 (0.00 B)
TensorFlow/Keras MSE: 19.1953
```



Actual VS Predicted Output

Inference

The scratch LSTM reaches a test MSE of 17.3962, and the Keras LSTM reaches 19.1953 — both in the same order of magnitude, and both, at 8 hidden units, only reach a modest final training loss (≈ 0.0105 in scaled units by epoch 900, still trending down at the end of training rather than fully plateaued). This is reasonable evidence that the scratch gate/BPTT implementation is functioning correctly: it is not just close to the library version, it is marginally better on this test split, so there’s no sign of a systematic bug like a missing gradient term or a sign error that would show up as much larger error.

The two numbers are not expected to match exactly, for reasons already visible in the code differences noted above: the scratch model trains with plain per-sample SGD ($\eta = 0.001$) while Keras trains with full-batch Adam for the same 1000 epochs, so the two are different optimization trajectories over the same architecture, not a strict apples-to-apples re-run. (**Assuming:** the ≈ 1.8 gap in MSE is attributable to this optimizer/batching difference plus the small random weight initialization.

5 CNN: Scratch Code and Tensor Flow Implementation

Problem Statement

The task is binary classification of brain MRI images into tumor (**yes**) versus no-tumor (**no**), using the `navoneel/brain-mri-images-for-brain-tumor-detection` Kaggle dataset. Each image is a 2D grid of pixel intensities, and the distinguishing signal — the tumor region’s shape, edges, and local intensity contrast — is spatial: it can appear at different positions within the scan and at different scales, so a plain fully-connected network reading raw flattened pixels would need to relearn the same pattern separately for every possible location. A CNN fits this problem because convolution applies the same small set of learned filters at every spatial position (weight sharing), giving the network some built-in robustness to where in the image the relevant feature sits, while max pooling then reduces spatial resolution and keeps only the strongest filter response in each region, discarding exact pixel-level position while retaining what was detected. The scratch (NumPy) implementation below builds this forward/backward pipeline by hand for the underlying calculus; the Keras version afterward implements the identical architecture with autodiff and a more robust optimizer, so the two can be compared directly. The full scratch implementation is available at `cnn.ipynb`.

Scratch Code

Loading the Data: Images as Pixel Arrays

```
img = Image.open(fpath).convert("L").resize((IMG_SIZE, IMG_SIZE))
images.append(np.array(img, dtype=np.float64))
```

`.convert("L")` maps each pixel to a single grayscale intensity (dropping any RGB channels), and `.resize` forces every image, regardless of its original dimensions, onto a fixed 32×32 grid. So each sample becomes

$$X^{(n)} \in \mathbb{R}^{32 \times 32}, \quad X_{i,j}^{(n)} = \text{intensity at row } i, \text{ column } j.$$

Stacking all $N = 253$ samples and their folder-derived labels (**yes** $\rightarrow 1$, **no** $\rightarrow 0$):

$$X \in \mathbb{R}^{N \times 32 \times 32}, \quad y \in \{0, 1\}^{N \times 1}.$$

Class Balance Check

```
n_pos = int(y.sum())
n_neg = len(y) - n_pos
```

Since $y_i \in \{0, 1\}$, summing the label vector directly counts positives:

$$n_{\text{pos}} = \sum_{i=1}^N y_i, \quad n_{\text{neg}} = N - n_{\text{pos}}, \quad p_{\text{pos}} = \frac{n_{\text{pos}}}{N}.$$

Here $n_{\text{pos}} = 155$, $n_{\text{neg}} = 98$, i.e. a 61.3%/38.7% split — skewed but not the $> 80\%$ majority that would force rebalancing or class weighting.

Train-Test Split (Stratified)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

As before, a random partition holds out 20% of rows, giving $X_{\text{train}} \in \mathbb{R}^{202 \times 32 \times 32}$ and $X_{\text{test}} \in \mathbb{R}^{51 \times 32 \times 32}$. The addition here is `stratify=y`: the partition is constrained so that the 61.3%/38.7% class ratio is preserved, within rounding, in both X_{train} and X_{test} , rather than left to an unconstrained random draw.

Standard Scaling

```
X_train_flat = x_scaler.fit_transform(X_train.reshape(n_train, -1))
X_test_flat = x_scaler.transform(X_test.reshape(n_test, -1))
```

`reshape(n_train, -1)` flattens each 32×32 image into a length-1024 vector so that `StandardScaler` treats each of the 1024 pixel positions as an independent feature column:

$$z_j = \frac{x_j - \mu_j}{\sigma_j}, \quad j = 1, \dots, 1024,$$

with μ_j, σ_j computed per pixel position from the training images only (`fit_transform`) and reused on the test images (`transform`). The result is reshaped back to $(32, 32)$ immediately after, so the convolution below still operates on a 2D grid.

CNN Hyperparameters and Output Dimensions

```
conv_out = IMG_SIZE - kernel_size + 1
pool_out = conv_out // pool_size
flat_size = pool_out * pool_out * num_filters
```

With a $k \times k$ kernel, stride 1, and no padding ("valid"), each spatial dimension shrinks as

$$\text{conv_out} = H - k + 1 = 32 - 3 + 1 = 30,$$

since a $k \times k$ window has nowhere left to slide once it's $k - 1$ pixels from the edge. Pooling with window size s and stride s then divides that down:

$$\text{pool_out} = \left\lfloor \frac{\text{conv_out}}{s} \right\rfloor = \left\lfloor \frac{30}{2} \right\rfloor = 15.$$

Flattening $f = 4$ channels of 15×15 feature maps gives the dense layer's input size:

$$\text{flat_size} = 15 \times 15 \times 4 = 900.$$

Weight Initialization

```
Wc = np.random.randn(num_filters, kernel_size, kernel_size) * 0.1
bc = np.zeros((num_filters, 1))
Wd = np.random.randn(1, flat_size) * 0.1
by_d = np.zeros((1, 1))
```

$$W_c \in \mathbb{R}^{f \times k \times k} = \mathbb{R}^{4 \times 3 \times 3}, \quad b_c \in \mathbb{R}^{4 \times 1}, \quad W_d \in \mathbb{R}^{1 \times 900}, \quad b_y \in \mathbb{R}^{1 \times 1}.$$

As with the RNN, weights are drawn from $\mathcal{N}(0, 1)$ and scaled by 0.1 to keep pre-activations small, avoiding early ReLU/sigmoid saturation; biases start at zero since the random filters already break symmetry between channels.

Forward Pass: Convolution + ReLU

```
patch = X[i:i + k, j:j + k]
for c in range(f):
    Z[i, j, c] = np.sum(patch * W[c]) + b[c, 0]
A = np.maximum(Z, 0)
```

For every output position (i, j) and filter c , the kernel is overlaid on the corresponding input patch and the elementwise product is summed (this is cross-correlation, what CNN libraries call "convolution," since the kernel isn't flipped):

$$Z_{i,j,c} = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} X_{i+p, j+q} W_{c,p,q} + b_c, \quad A_{i,j,c} = \max(Z_{i,j,c}, 0).$$

The same W_c is reused at every (i, j) — weight sharing, the convolutional analogue of the RNN reusing W_{xh} at every timestep.

Forward Pass: Max Pooling

```

window = A[i * size:i * size + size, j * size:j * size + size, c]
idx = np.unravel_index(np.argmax(window), window.shape)
P[i, j, c] = window[idx]
argmax[i, j, c] = idx

```

For each non-overlapping $s \times s$ window:

$$P_{i,j,c} = \max_{0 \leq p,q < s} A_{is+p, js+q, c}.$$

The (p^*, q^*) achieving that max is stored in `argmax`, since the max operation isn't differentiable in closed form but has a subgradient that routes entirely through whichever input attained it — needed for the backward pass below.

Forward Pass: Flatten, Dense, Sigmoid

```

Z = W @ F + b
A = 1 / (1 + np.exp(-Z))

```

`F = P1.reshape(-1, 1)` stacks the pooled feature maps into a single column $F \in \mathbb{R}^{900 \times 1}$. The dense layer then reads this out as one linear combination, squashed to a probability:

$$Z_2 = W_d F + b_y \in \mathbb{R}^{1 \times 1}, \quad A_2 = \sigma(Z_2) = \frac{1}{1 + e^{-Z_2}} \in (0, 1),$$

where A_2 is interpreted as the predicted probability of the tumor class.

Loss Function (Binary Cross-Entropy)

```

a = np.clip(A2, eps, 1 - eps)
loss = -(target * np.log(a) + (1 - target) * np.log(1 - a)).sum()

```

$$\mathcal{L} = -[y \log(a) + (1 - y) \log(1 - a)], \quad a = A_2.$$

Clipping a to $[\varepsilon, 1 - \varepsilon]$ avoids $\log(0)$ when floating-point rounding pushes the sigmoid output exactly to 0 or 1.

Output Layer Gradient

```

dZ2 = A2 - target
dW = dZ @ F.T
db = dZ
dF = W.T @ dZ

```

Differentiating $\mathcal{L}$ with respect to a and $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ together collapse, as with any sigmoid-plus-cross-entropy pairing, to:

$$\frac{\partial \mathcal{L}}{\partial Z_2} = A_2 - y = \delta_2,$$

the same clean form as $dy = \hat{y} - \text{target}$ in the RNN. Pushing this through $Z_2 = W_d F + b_y$:

$$\frac{\partial \mathcal{L}}{\partial W_d} = \delta_2 F^\top, \quad \frac{\partial \mathcal{L}}{\partial b_y} = \delta_2, \quad \frac{\partial \mathcal{L}}{\partial F} = W_d^\top \delta_2.$$

F here is the pooled activation *feeding into* the dense layer, not A_2 (the dense layer's own output) — using A_2 in place of F would be the backwards mistake.

Backward Through Max Pooling

```
di, dj = argmax[i, j, c]
dA[i * size + di, j * size + dj, c] += dP[i, j, c]
```

Since $P_{i,j,c} = A_{is+p^*, js+q^*, c}$ depends on exactly one entry of A within each window,

$$\frac{\partial P_{i,j,c}}{\partial A_{m,n,c}} = \begin{cases} 1 & (m,n) = (is+p^*, js+q^*) \\ 0 & \text{otherwise,} \end{cases}$$

so $\partial \mathcal{L} / \partial A$ is zero everywhere except at each window's stored max location, where it equals $\partial \mathcal{L} / \partial P$ at that window.

Backward Through Convolution

```
dZ1 = dA1 * (Z1 > 0)
patch = X[i:i + k, j:j + k]
for c in range(f):
    dW[c] += dZ[i, j, c] * patch
    db[c, 0] += dZ[i, j, c]
```

Using $\text{relu}'(z) = \mathbb{K}[z > 0]$:

$$\delta_{i,j,c}^{\text{conv}} = \frac{\partial \mathcal{L}}{\partial A_{i,j,c}} \odot \mathbb{K}[Z_{i,j,c} > 0].$$

Because W_c is reused at every output position, its gradient sums contributions over every position it touched, weighted by the corresponding *input* patch (not the conv layer's own output A or Z):

$$\frac{\partial \mathcal{L}}{\partial W_{c,p,q}} = \sum_{i,j} \delta_{i,j,c}^{\text{conv}} X_{i+p,j+q}, \quad \frac{\partial \mathcal{L}}{\partial b_c} = \sum_{i,j} \delta_{i,j,c}^{\text{conv}}.$$

Weight Update (Gradient Descent)

```
Wd -= learning_rate * dWd
by_d -= learning_rate * dbd
Wc -= learning_rate * dWc
bc -= learning_rate * dbc
```

$$W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial W}, \quad \text{for every } W \in \{W_c, b_c, W_d, b_d\}.$$

As in the RNN, this fires once per training image, so across 202 training samples this is stochastic gradient descent, repeated for 30 epochs.

Testing — Forward Pass Only

```
_, A1 = conv_forward(Xn, Wc, bc)
P1, _ = pool_forward(A1, pool_size)
F = P1.reshape(-1, 1)
_, A2 = dense_forward(F, Wd, by_d)
predictions.append(1 if A2[0, 0] >= 0.5 else 0)
```

Identical forward computation as training, with the trained weights and no gradient computation, applied to the 51 held-out images. The probability is thresholded at 0.5 to produce a hard label:

$$\hat{y} = \mathbb{K}[A_2 \geq 0.5].$$

Evaluation

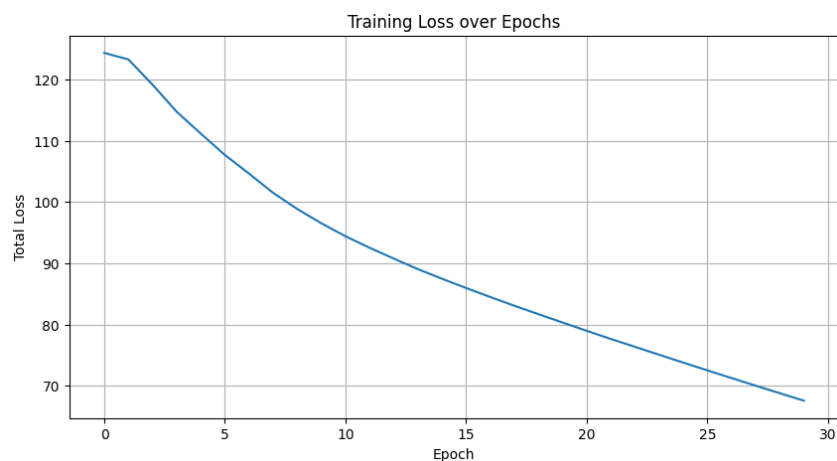
```
acc = accuracy_score(y_test, predictions)
```

$$\text{Accuracy} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \mathbb{K}[\hat{y}_i = y_i], \quad N_{\text{test}} = 51,$$

the fraction of held-out images correctly classified; the accompanying confusion matrix breaks this same count down into true/false positives and negatives.

Output

```
(253, 32, 32) (253, 1)
Tumor (1): 155 (61.3%) No tumor (0): 98 (38.7%)
Epoch 0 Loss = 0.615716
Epoch 5 Loss = 0.533183
Epoch 10 Loss = 0.467395
Epoch 15 Loss = 0.425540
Epoch 20 Loss = 0.390942
Epoch 25 Loss = 0.358976
Test Accuracy : 0.7255
Confusion Matrix:
[[12  8]
 [ 6 25]]
```



Actual VS Predicted Output

Keras Version — What's Different

Data loading, the stratified train-test split, and standard scaling are identical to the scratch version above, so they are not repeated here.

```
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(4, kernel_size=3, activation="relu",
                           input_shape=(IMG_SIZE, IMG_SIZE, 1)),
    tf.keras.layers.MaxPooling2D(pool_size=2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
```

These four layers implement exactly the equations above: **Conv2D** is $Z_{i,j,c} = \sum_{p,q} X_{i+p,j+q} W_{c,p,q} + b_c$ followed by ReLU, **MaxPooling2D** is the windowed max, **Flatten** is the reshape into F , **Dense(1, "sigmoid")** is $\sigma(W_d F + b_y)$. Keras computes $\partial \mathcal{L} / \partial W$ for every layer via reverse-mode autodiff rather than the explicit `*_backward` functions above, so there's no separate backward-pass math to write — it's the same chain rule, just traced automatically instead of by hand.

```
optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)
model.compile(optimizer=optimizer, loss="binary_crossentropy")
```

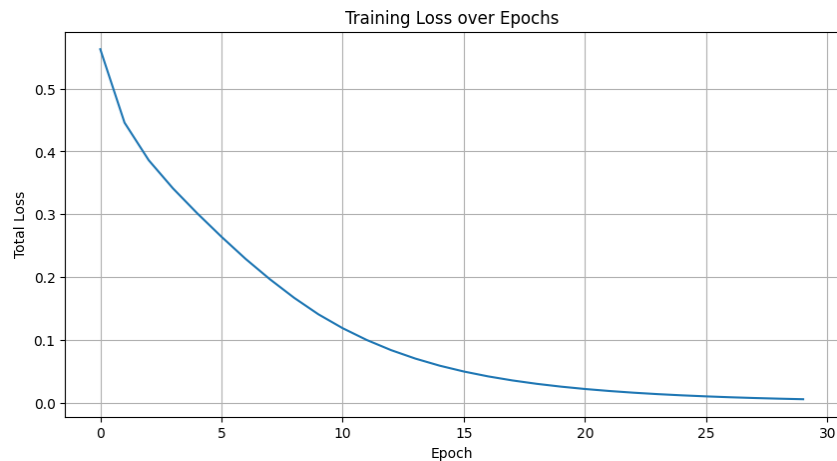
"binary_crossentropy" is the same $\mathcal{L} = -[y \log a + (1 - y) \log(1 - a)]$ used above. The optimizer differs: instead of the plain update $W \leftarrow W - \eta \partial \mathcal{L} / \partial W$, Adam keeps running estimates of the gradient's first and second moments, $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ and $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$, bias-corrects them, and updates as

$$W \leftarrow W - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}},$$

which adapts the effective step size per-parameter rather than using one fixed η for every weight, as the scratch version does.

Output using Keras

```
Epoch    0  Loss = 0.562588
Epoch    5  Loss = 0.264360
Epoch   10  Loss = 0.118933
Epoch   15  Loss = 0.050028
Epoch   20  Loss = 0.022224
Epoch   25  Loss = 0.010485
Test Accuracy : 0.8235
Confusion Matrix:
[[13  7]
 [ 2 29]]
```



Actual VS Predicted Output

Inference

The dataset itself is small and moderately imbalanced: $N = 253$ images (61.3% tumor, 38.7% no-tumor), split into 202 training and 51 test images with the class ratio preserved via stratification. With only 51 test images, each single misclassification moves the reported accuracy by roughly 2 percentage points ($1/51 \approx 1.96\%$), so accuracy alone is a coarse signal here — the confusion matrix breakdown matters more than the headline number for judging whether errors skew toward false negatives (missed tumors) or false positives.

Reading the two confusion matrices with rows as true class and columns as predicted class (order 0,1): the scratch model gives 8 false positives and 6 false negatives out of 51, while Keras gives 7 false positives and 2 false negatives. So Keras's higher test accuracy (0.8235 vs. 0.7255) is not just a uniform improvement — almost all of the gain comes from cutting missed tumors (false negatives) from 6 down to 2, while false positives barely move ($8 \rightarrow 7$). For this task, where a false negative means an undetected tumor, that asymmetry matters more than the 10-point accuracy gap on its own.

The architecture, loss, and per-sample update frequency (batch size 1, 30 epochs) are matched by construction between the two implementations, so the difference traces to the optimizer: plain fixed-step SGD in the scratch version versus Adam's per-parameter adaptive step size in the Keras version. The loss curves confirm this directly — at epoch 25 the scratch loss is still at 0.359, whereas Keras reaches 0.0105 over the identical 30 epochs, roughly a $34\times$ lower training loss. Adam's bias-corrected moment estimates let each weight take a larger effective step early on and shrink it as gradients stabilize, which explains both the faster loss decay and the corresponding drop in missed tumors on the test set.